

Connect your own tools



Three individual workshops. Your company, your accounts, your results.

Connect your own tools

Reviewed October 6, 2026.

Build two local pages first. For live email, add your own Google mailbox and exact work-email validation. Gifts, LinkedIn, publishing and ads are optional extensions.

You supply your own accounts. The shipped templates contain no keys, recipients or production account IDs.

The runtime does not consume an OpenAI, Anthropic or OpenRouter model key. Your separate AI coding agent uses its own signed-in account. Do not buy every provider in this reference.

1. What you need for each part

Part	Have ready	Paid API needed?
1: Build your foundation	An AI coding agent with file and web access; Verified Python 3.10+ selected by the startup launcher; official installer if missing; Your own company website; Browser or local HTML preview	No.
2: Run your ten-account email pilot	Own Google OAuth mailbox and calendar; Complete current exclusions snapshot; Fresh work-email validation; Current employer evidence; one research provider optional	ZeroBounce or a current supported validation receipt; One finder only if you need new people/emails
3: Operate the core workflow; optional extra channels	Same project, outcomes and owned connections from Part 2; Optional: Gojiberry for LinkedIn, funded Loop & Tie for gifts, owned Netlify site for publishing; Optional: Calendly/CRM/forms/Metadata only for the extension you use	No new paid API for the main lane; chosen channel costs apply only when connected

2. Where credentials go

Keep company context, skills, pages and private recipient inputs in company-motion/, beside code/. Keep filled config and tokens in a private directory outside both folders. Add their file paths under `credential_files` in your private config.

Set the private directory to mode 0700 and each actual config or token file to mode 0600. Use the package's `code/evidence/workshop-config.example.json` as the config contract. Keep filled config and credential files out of the participant ZIP.

`credentials.env.example` is an optional export-name reference. The adapter does not load `.env` files. A file that sits on disk does nothing until the process receives its variables or its private config references the token paths.

A secret unlocks an account connection. It does not prove ownership, grant an outreach budget, populate recipients, or remove a hold.

3. Connection sequence

1. Build and open your local company page. You need no provider key for this.
2. Connect your own Google mailbox and primary calendar. Inspect the returned mailbox identity.
3. Import your complete, current customer, deal, opt-out and relationship exclusions.
4. Add one research source only when you need it. Inspect current employer evidence.
5. Validate exact work emails. Confirm credit balance before paid calls.
6. Prepare the own-sender test and ten prospect emails for ten distinct companies. Read all eleven exact messages and pin their actual keys with approve-pilot.
7. Select the original approved self-test key with run --action-key KEY --limit 1. Read its exact Gmail Sent match. Then select only the ten pinned prospect keys. The eleven-message limit applies across days.
8. Optionally connect Gojiberry or Loop & Tie for the chosen extension. Confirm owned resources and funding. Dispatch each original approved channel key with --action-key, then reconcile that same key.
9. Connect existing Calendly, CRM and website-form sources when your company uses them.
10. Add Metadata creative generation or the supported Netlify publish method after the core email path works.

4. Read-only connection prompt

Copy this into the same workshop project:

Read the package README and my private config. Check only the connections needed for the current lesson.

Do not print tokens or send email, invitations, gifts, form submissions, or ad campaigns.

Read the provider account and the exact configured resources. Check that they belong to my company.

Show a short table: tool, what it enables, actual proof, and one missing setup item.

Keep failed and unused connections separate. Do not claim a connection from a saved file name.

Save the real read receipts privately. Continue building the lesson artifact when a provider is unavailable.

5. Provider reference

5.1. Google: Gmail, Calendar and optional Sheets

When: Required for the live email path. Parts 2, 3.

Code support: Supported in portable package.

Entering it enables: Send from your own approved mailbox, read the exact Sent message, read replies and exclude existing calendar meetings. Optional Sheets access reads your customer list.

Limits: A Google API key does not unlock private email. A downloaded OAuth client file alone is not a signed-in mailbox. This code reads calendars; it does not create appointments.

Credential: Desktop OAuth token JSON, or an existing delegated Workspace service-account JSON.

Config, input or setup fields: `credential_files.google`,
`credential_files[google:YOUR_EMAIL]` for each signed-in mailbox,
`policy.senders`, `history_since_epoch`, `settings.google_calendars.mailboxes`,
`settings.google_calendars.since`, `settings.revenue`.

Minimum account or credits: One Google account with Gmail and Calendar for the first pilot. Each personal OAuth sender signs in separately. Existing Workspace delegation needs an administrator. No purchased Google API credits are required by this code.

Setup:

1. Open Google Cloud Console and create your own workshop project.
2. Use APIs & Services, Library. Enable Gmail API and Google Calendar API. Enable Google Sheets API when reading a customer or revenue Sheet.
3. Use Google Auth platform, Branding. If not configured, choose Get Started. Enter app name, support email, audience and contact email, then complete the setup.
4. For an External app in Testing, use Audience, Test users, Add users. Add your own authorized mailbox.
5. Use Google Auth platform, Clients, Create Client. Select Desktop app, give it a name and create it. Download its JSON into your private credentials folder.
6. Run `python3 -B gtm.py setup-google --client-file /absolute/private/google-desktop-client.json`. Complete consent in your browser with the exact authorized sender.
7. The helper saves the signed-in credential privately and writes only its path back to your private config. Later configure retains the connection.
8. Inspect `connected_mailbox` and `outreach_sent:0`. Configure actual calendar, history and optional Sheet sources before releasing an email.

Connection proof: Gmail `/profile` returns your configured mailbox. A primary-calendar read succeeds. History pagination finishes for your configured baseline. A later authorized send counts only after an exact Sent match.

Provider wait: Google consent or administrator approval can delay setup. Workspace delegation can take up to 24 hours. External OAuth apps in Testing may issue seven-day refresh tokens. Reauthorize when revoked or expired.

Fallback: Finish company context, accounts, page and exact message preparation. Mark email sending blocked until your mailbox and calendar reads work.

Blank or structural example:

```
{
  "type": "authorized_user",
  "email": "",
  "client_id": "",
  "client_secret": "",
  "refresh_token": "",
  "scopes": [
    "https://www.googleapis.com/auth/gmail.modify",
    "https://www.googleapis.com/auth/gmail.send",
    "https://www.googleapis.com/auth/calendar.readonly",
    "https://www.googleapis.com/auth/spreadsheets.readonly"
  ]
}
```

Optional exported variable names: V5_GOOGLE_FILE, V5_GOOGLE_SERVICE_ACCOUNT_FILE.

Checks:

1. The downloaded Desktop client JSON and the resulting authorized_user JSON are different files.
2. Never paste the refresh token into a workshop prompt or PDF.
3. The Desktop helper currently requests all four scopes, including Sheets readonly. Sheets API is only needed when you use a sheet.
4. If a Google API returns disabled or permission denied, enable that exact API or fix its scope. A blocked company website is a different issue.
5. Repeat the consent helper for each authorized sender. It saves a mailbox-specific credential_files entry. Each credential can access only its own signed-in mailbox.
6. The sender cap is not the release approval. approve-pilot pins one exact self email and ten exact prospect emails across ten distinct companies. The prospect actions stay locked until the exact self Sent receipt exists.
7. Dispatch the exact original pinned self-test key with run --action-key ACTUAL_CONTROLLED_ACTION_KEY --limit 1, then reconcile that same key. A limit alone is global; the selector excludes unrelated approved grants and creates no new authority.
8. Google OAuth and delegated service-account files stay private at mode 0600 outside code/ and company-motion/. Runtime consumption checks the private path as well as configuration. Do not copy a service-account JSON into the downloaded runtime.

Official sources: [Google desktop OAuth](#), [Google OAuth token expiration](#), [Google delegation](#), [Google project creation](#), [Google API enabling](#), [Google OAuth screen and test users](#).

Local contract: tools/setup.py:setup_google, configure,
engine/channels/email.py:delegated_token, Mailbox,
engine/channels/meetings.py:calendars.

5.2. Your current customer, deal and opt-out list

When: Required before any live outreach. Parts 2, 3.

Code support: Current snapshot supported; strict complete-native mode supported for repeated operation.

Entering it enables: Remove customers, open deals, opt-outs and existing relationships. Snapshot mode uses a current owner-reviewed export. Native mode reads actual maintained customer, CRM, history and booking sources each cycle.

Limits: A blank export, partial range, stale source or unknown customer status cannot establish company-wide coverage. Native mode needs complete positive current receipts and does not fall back to a snapshot when its sources fail.

Credential: No API key for a current private owner-reviewed snapshot.

Config, input or setup fields: No config field for snapshot path; `import --kind exclusions --file PRIVATE_JSON, settings.exclusion_mode: snapshot or native, settings.native_exclusions, settings.revenue, settings.history_sources, settings.booking_sources, history_since_epoch`.

Minimum account or credits: For the classroom pilot: one complete current owner-reviewed export. For unattended native mode: a maintained complete customer Sheet, at least one CRM inventory, full mailbox history, every actual booking source and their required permissions.

Setup:

1. Export customers, open deals, opt-outs and active relationships from your own systems.
2. Resolve company domains and email addresses. Keep a reason for each exclusion.
3. Save the private snapshot JSON with `complete:true`, the real `observed_at`, a source description and all three coverage names.
4. Run `python3 -B gtm.py import --kind exclusions --file /absolute/private/exclusions.json`. Refresh the actual export when its source is older than 65 minutes.
5. Read the connection result. Resolve missing or ambiguous domains before selecting recipients.
6. For native mode, connect the complete maintained customer Sheet and at least one actual Salesforce or HubSpot source. Use the native setup below.
7. Set `settings.exclusion_mode:native` and `settings.native_exclusions` to revenue plus the actual CRM sources. Keep `history_since_epoch:0` for full mailbox history. Include any actual Instantly or Gijiberry history you rely on.
8. Run refresh and inspect each complete positive customer, deal, suppression, history and booking receipt. Resolve unknown status, missing domains, partial pagination or unsupported sources.
9. Keep snapshot mode for a manual pilot when the company cannot provide that complete native coverage. Refresh the real owner export before another live release.

Connection proof: Snapshot mode checks actual timestamp, declared source coverage and bytes. Native mode binds fresh complete customer/CRM/history/booking inventories to the selected accounts. Any failed or unresolved required source holds affected outreach.

Provider wait: Export timing and owner review depend on your systems. Live outreach waits for complete current coverage.

Fallback: Build assets and messages. Keep sending paused until you have a complete current list.

Blank or structural example:

```
{
  "complete": false,
  "observed_at": "",
  "source": "",
  "coverage": [
    "customers",
    "open_deals",
    "suppressions"
  ],
  "records": [
    {
      "entity_type": "domain",
      "value": "",
      "reason": ""
    }
  ]
}
```

Checks:

1. The shipped template deliberately starts complete:false.
2. Only the owner can confirm a complete source scope.
3. The package remembers the imported private file path and rereads its bytes and real timestamp each cycle. The timestamp must describe the underlying source read.
4. Native mode is a portable workshop addition with local tests. An actual native source connection remains an attendee test.
5. A complete maintained source is required. A header-only or filtered range cannot honestly cover all company customers.
6. Do not update a source timestamp to hide old data. Read the actual maintained systems.

Local contract: tools/setup.py:snapshot,import_records,refresh_portable,
engine/validation.py:account_domains.

5.3. ZeroBounce

When: Required for fresh validation on the email path. Parts 2, 3.

Code support: Balance reads, exact-address validation and stale-complete renewal supported.

Entering it enables: Check the exact work email. Store its returned status and actual checked time.

Limits: Validation does not prove current employment, permission to send, absence of a deal, or delivery.

Credential: API key text.

Config, input or setup fields: credential_files.zerobounce, person.email_verification or zb_status/zb_checked_at.

Minimum account or credits: An active credit balance for the distinct addresses you validate. Budget at least 11 lookups for ten prospects and one controlled check, plus replacements. Provider rules determine actual billing.

Setup:

1. Create or use your own ZeroBounce account.
2. Copy the API key from your account into a private text file.
3. Add its path under `credential_files.zerobounce`.
4. Run `python3 -B gtm.py research --operation zerobounce-balance`. Read actual `credits_remaining` before validating addresses.
5. Check your credit balance with the provider's read-only balance endpoint.
6. Validate one exact address. Save the native response with its checked time.
7. Proceed only when the address matches and the status is valid. Review catch-all, invalid and unknown results.

Connection proof: Returned address matches the requested email and status is valid. The package requires validation evidence no older than seven days.

Provider wait: The official validator reports one to 30 seconds per address. Unknown results remain unqualified in this package.

Fallback: Use your own current provider validation receipt with the exact email, provider, valid status and `checked_at`. Never invent a valid result.

Blank or structural example:

```
{
  "email": "",
  "status": "valid",
  "provider": "zerobounce",
  "checked_at": ""
}
```

Optional exported variable names: `V5_ZEROBOUNCE_FILE`, `V5_ZEROBOUNCE_TOKEN`.

Checks:

1. The adapter sends the key in the API-required query. It excludes that key from the saved payload.
2. No automatic replay follows an uncertain validation request.
3. An exact valid receipt must be current within seven days. The same operation reuses a fresh receipt and renews a known completed stale lookup without erasing prior receipts.
4. A prior uncertain lookup cannot renew or replay. Resolve that original attempt first.
5. Never type valid into a person record as a substitute for the actual validation result.

Official sources: [ZeroBounce validation](#), [ZeroBounce credit balance API](#).

Local contract: `engine/research.py:validate_email`,
`engine/validation.py:person_issues`, `engine/research.py:validation_balance`.

5.4. Sixtyfour

When: Optional research choice. Parts 2, 3.

Code support: Search and professional email discovery supported.

Entering it enables: Find companies or people with bounded filters. Find a professional email for a sourced person.

Limits: The local adapter does not expose Sixtyfour's full intelligence, phone, monitor or workflow APIs. Its email lookup has `verify_emails:false`, so separate validation is required.

Credential: API key text.

Config, input or setup fields: `credential_files.sixtyfour`.

Minimum account or credits: The reviewed pricing lists 64 free credits. A ten-result search plus ten professional email discoveries costs up to six credits under the adapter's reservations: 1 + 5. This excludes other lookups, validation and unused results.

Setup:

1. Create your own Sixtyfour organization.
2. Open Settings, API Keys and create a key with an expiration.
3. Save it privately. Set `credential_files.sixtyfour`.
4. Set your own monthly allowance and explicitly enable paid research.
5. Search with mode company or people, `page_size` equal to `max_results`, `output_shape` raw and `simple_filters`.
6. Inspect each identity. Use PROFESSIONAL email discovery only for a sourced person, then validate the exact work email.

Connection proof: The native response has the requested entity or exact work-email candidate. Record source and retrieval time. Research output remains a candidate until identity checks pass.

Provider wait: Some provider jobs can return pending. Preserve the request and handle. An unsupported or missing native read handle needs reconciliation, not another paid submission.

Fallback: Use ten known target companies and current official company/person sources. If credits run out, finish the verified subset and report the count.

Blank or structural example:

```
{
  "mode": "people",
  "page_size": 10,
  "max_results": 10,
  "output_shape": "raw",
  "simple_filters": {}
}
```

Optional exported variable names: `V5_SIXTYFOUR_FILE`, `V5_SIXTYFOUR_TOKEN`.

Checks:

1. The source engine's 15,000-credit ceiling belongs to Metadata. Set your own allowance.
2. Credits found in the account do not grant authority to spend them.

3. Paid research is paused by default. Use `research-on --sixtyfour-credits` with your own ceiling. This enables research only, not outreach.

Official sources: [Sixtyfour API keys](#), [Sixtyfour credit costs](#), [Sixtyfour filter search](#), [Sixtyfour email discovery](#).

Local contract: `engine/research.py:sixtyfour`.

5.5. Exa

When: Optional official-source research. Parts 1, 2, 3.

Code support: Search supported in portable package; original source had credential plumbing only.

Entering it enables: Find current web sources for a company, product or buying signal. Read linked official sources to support the message.

Limits: Search snippets do not verify employment or email delivery. Exa does not send outreach.

Credential: API key text.

Config, input or setup fields: `credential_files.exa`.

Minimum account or credits: One account with search access and enough available usage for the searches you select. Confirm your dashboard's live allowance. No paid Exa account is required for Part 1 when your agent already has web access.

Setup:

1. Create an Exa API key in your own dashboard.
2. Save the key privately and set `credential_files.exa`.
3. Enable only the research you intend to pay for.
4. Run one narrow company or official-domain query.
5. Read the source page. Save the URL, `observed_at` and the claim it supports.

Connection proof: A source URL answers the intended question, and the page's content supports the recorded claim. A search response alone is only discovery.

Provider wait: The API may apply rate limits. Respect the returned wait. Do not repeat an uncertain paid request.

Fallback: Read the company's official website with the agent's web tool and save exact source evidence.

Blank or structural example:

```
{
  "query": "",
  "numResults": 3
}
```

Optional exported variable names: `V5_EXA_FILE`, `V5_EXA_TOKEN`.

Checks:

1. Run `research --operation exa --input PRIVATE_JSON` after `research-on`. The exact input needs a nonempty query and `numResults` from 1 to 10.
2. The portable package adds this search call. It does not claim that the source engine already had it.
3. Paid research is paused by default. Use `research-on --sixtyfour-credits` with your own ceiling. This enables research only, not outreach.

Official sources: [Exa search contract](#), [Exa pricing](#).

Local contract: `engine/research.py:exa, tools/setup.py:research`.

5.6. Apify

When: Optional profile and public-signal source. Parts 2, 3.

Code support: Five allowlisted actors supported.

Entering it enables: Read public LinkedIn person/company data and recent public comments using the selected actor. Save the accepted run ID and read its dataset.

Limits: A scraped comment is not buying intent or authorization. This adapter does not post, invite or message on LinkedIn.

Credential: API token text.

Config, input or setup fields: `credential_files.apify`, `settings.radar`.

Minimum account or credits: Enough account usage for one selected actor run. Check that actor's current price and input schema. Actor fees and platform usage vary, so the guide does not promise a fixed dollar allowance.

Setup:

1. Create or use your own Apify account.
2. Copy your token from Console, Integrations, into a private file.
3. Choose one allowlisted actor. Read its current input schema and price.
4. Use one known profile or a narrow tracked-profile/keyword scan. Launch once.
5. Save `run.id`. Poll that same run until `SUCCEEDED`.
6. Read the full default dataset. Bind each accepted comment to its native post and author before researching the person.

Connection proof: The saved run ID matches the completed run. Its dataset is fully read. An accepted signal has the native comment, post, profile and actual comment time.

Provider wait: Actor runs are asynchronous. The source adapter requests a 1,800-second actor timeout. Keep the same run ID while waiting; that timeout is not a promised completion time.

Fallback: Use a current public post/comment URL and the exact text you can read. Mark signal acquisition manual. Do not simulate an actor result.

Blank or structural example:

```
{
  "allowed_actors": [
    "harvestapi-linkedin-profile-scraper",
    "harvestapi-linkedin-company",
    "pipelinelabs-leads-finder-with-emails-apollo-lusha-zoominfo",
    "harvestapi-linkedin-profile-posts",
    "harvestapi-linkedin-post-search"
  ]
}
```

Optional exported variable names: V5_APIFY_FILE, V5_APIFY_TOKEN.

Checks:

1. The adapter rejects comments older than 24 hours for its Radar path.
2. Relevance and current identity are still separate checks.
3. Paid research is paused by default. Use `research-on --sixtyfour-credits` with your own ceiling. This enables research only, not outreach.

Official sources: [Apify API authentication and runs](#), [Run and retrieve data](#).

Local contract: `engine/research.py:apify_start, apify_read, Signals`.

5.7. Triguna

When: Optional identity lookup. Parts 2, 3.

Code support: Person and company lookup supported.

Entering it enables: Read a person profile or company record from an exact existing native identifier.

Limits: There is no people-search method in this adapter. You need the LinkedIn URL or identifier first.

Credential: API key text.

Config, input or setup fields: `credential_files.triguna`.

Minimum account or credits: The reviewed official site lists five signup credits and one credit per answered lookup. Ten person lookups plus ten company lookups need up to 20 credits. Confirm balance before using this option.

Setup:

1. Get your own key from the Triguna app.
2. Save it privately and set `credential_files.triguna`.
3. Use the exact `profile_id` or `company_id` from a source.
4. Read the returned identity and current experience. Keep `data-source` and `fetches-time` evidence.
5. Map the actual employer, role and profile into the workshop's `current-identity` fields.

Connection proof: The returned person matches the profile, employer and role you selected. Source age is inspected. A lookup does not automatically mark current employment confirmed.

Provider wait: Follow provider restrictions and Retry-After. A failed or uncertain paid lookup is not automatically replayed.

Fallback: Use current official person/company sources or another supported identity source.

Blank or structural example:

```
{
  "profile_id": "",
  "include_network_details": false,
  "use_cache": false
}
```

Optional exported variable names: V5_TRIGUNA_FILE, V5_TRIGUNA_TOKEN.

Checks:

1. Some official documentation pages were unavailable to the review browser. Exact request names here come from local code.
2. Titles can be nested in positions. Do not infer the latest role from the first company label.
3. Paid research is paused by default. Use research-on --sixtyfour-credits with your own ceiling. This enables research only, not outreach.

Official sources: [Triguna official API and pricing overview](#).

Local contract: `engine/research.py:profile,company`.

5.8. MoltSets

When: Optional people and work-email source. Parts 2, 3.

Code support: Three tools supported.

Entering it enables: Search people, look up a business email from LinkedIn, or look up a mobile number.

Limits: The workshop needs no mobile number. The adapter exposes only `search_people`, `linkedin_to_business_email` and `linkedin_to_mobile_phone`. A risk score is not the package's fresh valid email verdict.

Credential: API key text.

Config, input or setup fields: `credential_files.moltsets`.

Minimum account or credits: Use available free search access or your own paid plan. The reviewed FAQ lists paid plans from \$27/month and separate phone-token subscriptions. Check current entitlement before calling; phone lookup is optional.

Setup:

1. Create a key under API Keys in your own MoltSets app.
2. Save it privately and set `credential_files.moltsets`.
3. Use exact title and `company_domain` filters for a people search.
4. Read status, source date, employer and work email. `Not_found` is a valid empty result.
5. Save current identity evidence and validate the exact email separately.

Connection proof: The response status is ok or not_found. A found record matches the intended employer and role. The HTTP client supplies the required User-Agent header.

Provider wait: Observe account fair-use and provider rate limits. A not_found response does not justify repeated paid calls.

Fallback: Use your already sourced people and official employer pages.

Blank or structural example:

```
{
  "operation": "search_people",
  "inputs": {
    "company_domain": "",
    "title": "",
    "limit": 10,
    "offset": 0
  }
}
```

Optional exported variable names: V5_MOLTSETS_FILE, V5_MOLTSETS_TOKEN.

Checks:

1. The local API uses a Bearer key. A Claude MCP connector's OAuth credentials are a separate connection.
2. Paid research is paused by default. Use research-on --sixtyfour-credits with your own ceiling. This enables research only, not outreach.

Official sources: [MoltSets API setup](#), [MoltSets people-search contract](#), [MoltSets account and token plans](#).

Local contract: engine/research.py:moltsets.

5.9. RocketReach

When: Optional contact lookup. Parts 2, 3.

Code support: Exact profile lookup and same-handle status check supported.

Entering it enables: Look up a known LinkedIn profile and read the status of that same native person handle.

Limits: The adapter does not expose a full search workflow. It does not send mail or validate your sender.

Credential: API key text.

Config, input or setup fields: credential_files.rocketreach.

Minimum account or credits: An account with API lookup entitlement and sufficient credits. The current public API documentation could not be read in this review, so no plan or price is promised.

Setup:

1. Obtain an API key from your own API-enabled RocketReach account.
2. Save it privately and set credential_files.rocketreach.

3. Look up the exact sourced LinkedIn URL once.
4. Save the returned native id. If pending, check that same id.
5. Use the result only when status is complete and the returned profile matches.

Connection proof: A complete result has the same LinkedIn URL. The package records the native lookup handle and response.

Provider wait: Pending is an expected provider state. Recheck the original handle, never launch a duplicate lookup.

Fallback: Use another supported finder or already verified work email.

Blank or structural example:

```
{  
"linkedin_url": ""  
}
```

Optional exported variable names: V5_ROCKETREACH_FILE, V5_ROCKETREACH_TOKEN.

Checks:

1. API entry point was unavailable to the browser on 2026-10-05. Setup details beyond the local contract remain unverified.
2. Paid research is paused by default. Use research-on --sixtyfour-credits with your own ceiling. This enables research only, not outreach.

Official sources: [RocketReach API entry point](#).

Local contract: engine/research.py:rocketreach.

5.10. Instantly

When: Optional existing-history and email-finder source. Parts 2, 3.

Code support: Selected enrichment and optional complete history reads supported.

Entering it enables: Read past Instantly emails to avoid duplicate outreach. Use an available Instantly-funded finder from its actual provider registry.

Limits: This adapter does not send through Instantly. A key does not make the portable setup read its history or make an unavailable finder usable. Select only a live funded provider from its registry.

Credential: API v2 key text.

Config, input or setup fields: credential_files.instantly,
settings.history_sources.instantly.

Minimum account or credits: Your own workspace with API v2 scopes for email history, provider registry and billing/enrichment reads. Paid finder calls require available workspace credits at the registry's actual cost.

Setup:

1. Create a scoped API v2 key in your own workspace.
2. Save it privately and set credential_files.instantly.

3. Set settings.history_sources.instantly:true only when this is an actual company outreach source. Native exclusion mode requires complete coverage of all relied-on history.

4. Read the complete email inventory with pagination.

5. For enrichment, inspect the provider registry and plan details first. Pick one available instantly_key-funded action.

6. Supply sourced name fields, exact profile and domain. Keep the result and validate the exact work email.

Connection proof: History pagination finishes. A finder reports the actual available credit balance and action cost. Missing required sourced input returns projection_omitted, not a guessed name.

Provider wait: Provider jobs can be pending. v5 retains unknown outcomes and account restrictions. Read-only history is subject to provider rate limits.

Fallback: Use Gmail history only if it fully covers your actual outreach. Otherwise export the missing past activity and keep affected recipients held.

Blank or structural example:

```
{
  "allowed_finders": [
    "bettercontact",
    "contactout",
    "findymail",
    "icypeas",
    "leadmagic",
    "prospeo",
    "wiza"
  ]
}
```

Optional exported variable names: V5_INSTANTLY_FILE, V5_INSTANTLY_TOKEN.

Checks:

1. Connecting Google to Instantly is not the same as connecting the workshop's own Gmail adapter.

2. Paid research is paused by default. Use research-on --sixtyfour-credits with your own ceiling. This enables research only, not outreach.

Official sources: [Instantly API introduction](#), [Instantly API documentation index](#).

Local contract: engine/research.py:instantly_registry,instantly_email,
engine/channels/email.py:Replies.instantly.

5.11. Calendly

When: Optional when your company uses it. Parts 3.

Code support: Booking and cancellation reads supported.

Entering it enables: Read exact bookings and cancellations for your selected event types. Exclude booked people and accounts from cold outreach.

Limits: The adapter does not create a Calendly event, choose a time, send a scheduling link, or verify attendance. Reading a booking does not establish that a meeting happened.

Credential: Personal access token text.

Config, input or setup fields: `credential_files.calendly`,
`settings.calendly_identity.email`, `settings.calendly_identity.user_uri`,
`settings.calendly_identity.organization_uri`, `settings.calendly_identity.since`,
`settings.calendly_identity.event_type_uris`.

Minimum account or credits: Calendly read API access is available on all plans in the reviewed official overview. Organization-wide data needs owner/admin scope. No paid scheduling API is used here.

Setup:

1. Open Calendly Integrations, API & Webhooks and generate a named personal token.
2. Save it privately and set `credential_files.calendly`.
3. Read `/users/me`. Capture your exact email, user URI and organization URI.
4. Read the event-type inventory. Select your actual demo event-type URIs.
5. Set a real history baseline since and read scheduled events plus every invitee page.
6. Inspect booked and canceled states. Keep attendance unknown.

Connection proof: Authenticated owner matches config. All selected event types exist. Every event and invitee belongs to the expected owner/program and the inventory finishes.

Provider wait: New bookings appear only after the recipient schedules. Token permissions and organization role can restrict coverage.

Fallback: Use your own calendar coverage if it captures all existing booked relationships. Otherwise keep the missing booking source held.

Blank or structural example:

```
{
  "email": "",
  "user_uri": "",
  "organization_uri": "",
  "since": "",
  "event_type_uris": []
}
```

Optional exported variable names: `V5_CALENDLY_FILE`, `V5_CALENDLY_TOKEN`.

Checks:

1. Entering the token without owner and event-type settings is incomplete.

Official sources: [Calendly personal tokens](#), [Calendly plan and role access](#).

Local contract: `engine/channels/meetings.py:calendly`.

5.12. HubSpot

When: Optional when your company uses it. Parts 2, 3.

Code support: Open deals, demo-form submissions and raw meeting-outcome reads supported.

Entering it enables: Read open deals and associated company domains. Read existing demo-form submissions. Record native meeting outcome rows.

Limits: The adapter does not create/update CRM deals or submit forms. Meeting attendance stays unknown because the exact meeting/contact association join is missing.

Credential: Scoped private-app access token text.

Config, input or setup fields: `credential_files.hubspot`,
`settings.hubspot_demo_forms.ids`, `settings.hubspot_demo_forms.since`,
`settings.attendance_since`.

Minimum account or credits: An account and private app with access to your actual deals, companies, forms and meeting data. Configure read scopes for each used endpoint. Do not buy a higher plan before checking the actual endpoint permission.

Setup:

1. Create or use a private app in your own HubSpot account.
2. Grant only endpoint scopes needed for deals/companies, forms and optional meeting reads.
3. Save the access token privately and set `credential_files.hubspot`.
4. Read pipelines and deals. Join each open deal to its actual company domain.
5. For forms, read the form inventory and set only your actual demo form ids and baseline.
6. Test a read of each configured source. Repair `MISSING_SCOPES` from the named endpoint's official documentation.

Connection proof: Full pipeline/deal/company pagination completes. Configured forms appear in native inventory. A submission includes its exact `conversionId`, email and `submittedAt`.

Provider wait: Native permissions, form access and associated-domain resolution can delay readiness. New form submissions are recipient actions.

Fallback: Use a current owner-reviewed complete exclusion export. If your website uses another form provider, do not claim HubSpot covers it.

Blank or structural example:

```
{
  "ids": [],
  "since": ""
}
```

Optional exported variable names: `V5_HUBSPOT_FILE`, `V5_HUBSPOT_TOKEN`.

Checks:

1. Local source uses forms v3 inventory and a legacy submissions route. Confirm endpoint access with a read before class.
2. An unclassified deal stage is conservatively excluded.

Official sources: [HubSpot private apps](#), [HubSpot forms guide](#).

Local contract: `engine/channels/meetings.py:forms, hubspot_deals, attendance`.

5.13. Salesforce

When: Optional when your company uses it. Parts 2, 3.

Code support: OAuth and open-opportunity reads supported after portable owner config.

Entering it enables: Read open opportunities and their exact account websites. Exclude those company domains.

Limits: The adapter does not write opportunities or infer pipeline revenue. A CRM connection alone does not prove the engine caused a deal.

Credential: Private text file with `client_id`, `client_secret`, `refresh_token` and `instance_url`.

Config, input or setup fields: `credential_files.salesforce`,
`settings.salesforce_owner_email`.

Minimum account or credits: A Salesforce user with API access and permission to read Opportunity and Account fields. Use an External Client App or an existing Connected App. New Connected App creation is restricted from Spring 2026.

Setup:

1. Have your Salesforce administrator create an External Client App for this private integration.
2. Authorize the API and refresh-token scopes for the intended user.
3. Save the four required values in a private file. Set `credential_files.salesforce` and the exact owner email.
4. Exchange the refresh token. Verify the returned instance and identity.
5. Read all open opportunities and the linked `Account.Name` and `Account.Website` fields.
6. Resolve missing account domains before relying on the exclusion coverage.

Connection proof: The authenticated instance and owner match config. Native query `done:true` is reached and returned row count matches `totalSize`. Unresolved domains remain visible.

Provider wait: Administrator approval and OAuth configuration may take longer than the workshop. API entitlement is an account prerequisite.

Fallback: Use a complete current owner-reviewed export of open deals and customers.

Blank or structural example:

```
{
  "client_id": "",
  "client_secret": "",
  "refresh_token": "",
  "instance_url": ""
}
```

Optional exported variable names: `V5_SALESFORCE_FILE`.

Checks:

1. The original source restricted identity to Metadata. The portable copy must match your configured owner.

Official sources: [Salesforce OAuth and External Client Apps](#).

Local contract:

engine/channels/meetings.py:_salesforce,_sf_query,salesforce_deals.

5.14. Your current customer Sheet or revenue Sheet

When: Optional native customer source. Parts 2, 3.

Code support: Customer-domain status format and original revenue schema supported.

Entering it enables: Read the complete customer Sheet and exclude exact current-customer domains. The original revenue schema also reads positive recognized revenue relationships without proving current-customer status.

Limits: A Sheets key alone cannot maintain customer status. Use actual maintained statuses, not guessed revenue. The source revenue layout is specific. A generic CSV or partial range is not automatically a complete customer source.

Credential: The same private Google OAuth credential, with Sheets read scope.

Config, input or setup fields: settings.revenue.mailbox, settings.revenue.sheet_id, settings.revenue.range, settings.revenue.format: customer_domains for the portable status format.

Minimum account or credits: Read access to your own sheet. Its headers and current-month column must match the adapter contract. No separate Sheets API key.

Setup:

1. Use your actual complete maintained customer source and the authorized Google mailbox.
2. Enable Sheets API. Grant Sheets readonly through the Google helper and check access to the exact private Sheet.
3. For the portable customer format, use exactly name, domain, status as the first row. Each subsequent row must have exactly three fields.
4. Use only actual status current_customer or former_customer. Unknown states hold the native inventory. Never label customers from invented or inferred revenue.
5. Set mailbox, sheet_id, a canonical all-column range such as your actual customer sheet A:C, and format:customer_domains privately. Row-bounded ranges are held in native mode.
6. Run native refresh. Inspect current_customers, rows_read, source identity and unresolved-domain results.
7. Use the original revenue schema only when the source actually has Account, Previous ARR, ARR, MRR and one numeric date column for the current month. It remains a revenue relationship source.

Connection proof: The actual configured Sheet returns its complete range, recognized column contract and customer statuses. Exact current_customer domains are excluded. Unknown status or unresolved identity blocks native coverage.

Provider wait: Your finance team's current month and domain mapping may be the limiting step.

Fallback: Use a complete current owner-reviewed customer export with domains. Do not force your company's revenue file into Metadata's schema without an explicit mapping.

Blank or structural example:

```
{
  "mailbox": "",
  "sheet_id": "",
  "range": "",
  "format": "customer_domains",
  "required_columns": [
    "name",
    "domain",
    "status"
  ],
  "allowed_actual_statuses": [
    "current_customer",
    "former_customer"
  ]
}
```

Optional exported variable names: V5_GOOGLE_FILE.

Checks:

1. The config key remains settings.revenue for compatibility, even when format:customer_domains contains customer status rather than revenue.
2. The source status Sheet must cover all current company customers. It is not a ten-prospect sample.
3. A source receipt proves what was read. The owner must supply a genuinely maintained source and complete range.
4. Former status does not remove an existing exclusion automatically. Exclusion removal remains a separate owner review.
5. Native mode rejects a nonzero mailbox history boundary and a row-bounded customer range. Do not weaken those checks to make setup pass.

Official sources: [Sheets values API](#).

Local contract: engine/channels/meetings.py:customers.

5.15. Loop & Tie

When: Required for the gift extension. Parts 3.

Code support: Meeting-gated email gift creation and native readback supported.

Entering it enables: Create one exact email-delivered gift and read its native creation/sent state. Require a verified meeting scheduler gate.

Limits: A token alone cannot fund a gift, set its meeting gate, provide a native numeric scheduler binding, or prove redemption. This adapter does not send postal mail.

Credential: Bearer access token text.

Config, input or setup fields: credential_files.loop_and_tie, settings.loop_and_tie_team, settings.gift_gate, policy.gift_fulfillment, approve-scope action_keys, limits and gift_budget.

Minimum account or credits: An API-enabled account, owned team, funded collection, required shipping permission and verified scheduler. Check the live collection price and balance. Public docs say to contact Loop & Tie support to register an integration.

Setup:

1. Arrange API access with Loop & Tie for your own account.
2. Complete the provider's OAuth flow and save the returned bearer token privately. The adapter does not refresh it automatically.
3. Read your actual team, schedulers, gift inventory and intended collection.
4. Set `loop_and_tie_team` to your owned team identifier. Verify its balance and permissions.
5. Use GIFT-GATE-HELPER.md. Capture the actual signed-in scheduler Edit page and capture time. For scheduler-edit-v1, match its exact URL and POST action, numeric form, form name and meeting URL to the owned native scheduler. Read the visible provider statement that redemption requires a meeting. Import within 65 minutes. Do not invent missing form controls.
6. Review recipient, collection, native currency, gift value, meeting requirement, expiry and exact copy. Approve one exact action with `approve-scope`, a gift count and explicit per-gift/total budget.
7. Dispatch the original approved gift action with `run --action-key ACTUAL_APPROVED_GIFT_ACTION_KEY --limit 1`. Reconcile that same key and read the native gift ID/sent event. The selector adds no approval and clears no hold.

Connection proof: The current scheduler-edit-v1 contract binds the exact scheduler Edit URL, same POST form action, numeric form, enabled name and meeting URL inputs, and visible meeting requirement to the authenticated owned native catalogue. The older input-control contract remains supported only when both exact controls exist. Sending rechecks capture freshness, funding and exact approved gift. Only a native sent event counts as sent.

Provider wait: API access and funding can take time. The owned scheduler capture must be no more than 65 minutes old when imported and sent, and cannot be dated in the future. Recapture the actual current form when stale. Gift email, redemption and meeting booking have separate waits.

Fallback: Produce the exact gift proposal and preview. Leave gifting blocked until funding, owned gate and native binding are verified.

Blank or structural example:

```
{
  "ui_contract": "scheduler-edit-v1",
  "external_id": "",
  "numeric_id": null,
  "name": "",
  "source": "",
  "meeting_required": false,
  "meeting_required_field": "",
  "external_id_field": "",
  "form_html_file": "",
  "capture_url": "",
  "captured_at": "",
  "provider": "loop_and_tie",
  "team_id": ""
}
```

Optional exported variable names: V5_LOOP_AND_TIE_FILE, V5_LOOP_AND_TIE_TOKEN.

Checks:

1. Gifting can be taught without claiming a paid gift was sent.
2. Set meeting_required:true only after the actual provider page check passes. The blank template leaves it false. Import makes authenticated read-only team and scheduler calls; it sends no gift.
3. If the browser cannot capture the actual form, keep the reviewed gift preview and repair browser access. Do not fill numeric_id from the API external ID.
4. Use a gift count and per-gift/total value ceiling. Loop & Tie credit-priced collections use USD: one credit equals one USD. The adapter uses that documented contract only for native collection records with a finite price; it preserves any explicit provider currency. The resulting currency must match the grant. Other unknown currency stays held.
5. Use capture_url from the actual authenticated HTTPS loopandtie.com page or its actual subdomain. provider is loop_and_tie; team_id must match your configured own team. captured_at is the actual capture time, not import time. The source HTML and those fields are bound to the native read receipt.
6. Use scheduler-edit-v1 for the current Edit Calendar page. The old meeting_required_field and external_id_field names are needed only when those real controls exist inside the same numeric form. Never invent a missing control or copy a field from an example.

Official sources: [Loop & Tie OAuth and integration access](#).

Local contract: engine/channels/gifts.py:catalogue, send, reconcile,
tools/setup.py:import_records gift-gate.

5.16. Gojiberry

When: Required for the LinkedIn extension. Parts 3.

Code support: Paused contact preparation and exact contact/message release supported; campaign activation stays in owned UI.

Entering it enables: Prepare inert contacts in an existing owned list. Release only an exact approved contact and read native queued/sent state. Send an individually approved

existing-thread message when its native prerequisites pass.

Limits: A key does not connect a LinkedIn seat, create a list or activate a whole campaign. The runtime holds campaign activation because it could release unrelated contacts. Queue state does not prove delivery or acceptance.

Credential: API key text.

Config, input or setup fields: `credential_files.gojiberry`, `settings.social_owned`, `settings.social_prepared`, `settings.social_reply_grants`, `policy.linkedin_dispatch`.

Minimum account or credits: An API-enabled Gojiberry account with your own connected LinkedIn seat and permitted capacity. Price and safe capacity depend on your account; the code does not promise a universal plan or daily allowance.

Setup:

1. Get your own key at Gojiberry Settings, API. Save it privately.
2. Connect your own LinkedIn seat in Gojiberry. Create an isolated owned list and inactive invitation-only campaign in the native app.
3. Read the actual campaign/list/seat IDs, name and exact steps into `settings.social_owned`. No automatic message steps are allowed in this invitation exercise.
4. Use guarded preparation to create the exact contact in that owned list. The code forces `paused` and `readyForCampaign:false`. Bind actual person/profile evidence.
5. Inspect all lists and contacts in the isolated campaign. After explicit owner review, activate only that isolated campaign in the provider UI while prepared contacts stay paused. The runtime never activates the whole campaign.
6. Prepare and review the exact contact action. Approve its action key with `approve-scope` and an explicit social count.
7. Release only the original exact paused-contact key with `run --action-key ACTUAL_APPROVED_SOCIAL_ACTION_KEY --limit 1`. Reconcile that same key and read actual `campaignStatus`. Keep queued separate from sent, invitation accepted and message delivered.
8. A new message requires native invitation acceptance and its exact isolated step, or an individually authorized existing-thread message.

Connection proof: Owned seat, campaign steps, list, contact identity and exact message match. Native queued contact means queued. A matching provider sent event or `deliveredAt/backend` message evidence is needed for sent.

Provider wait: Native queue and recipient waits are separate. For an ambiguous provisioning request, inspect the original attempt and owned provider list/contact before any new write. An exact outreach action uses `reconcile`; uncertain provisioning has no CLI reconciliation command.

Fallback: Write the exact LinkedIn invitation/message and source evidence. Mark it prepared. Leave release held until owned resources and native readiness exist.

Blank or structural example:

```
{
  "sender": {
    "campaign_id": "",
    "list_id": "",
    "seat_id": "",
    "name": "",
    "steps": []
  }
}
```

Optional exported variable names: V5_GOJIBERRY_FILE, V5_GOJIBERRY_TOKEN.

Checks:

1. A paused contact from a manual campaign cannot be adopted as workshop-owned prepared work without its exact ownership record.
2. The adapter has no browser fallback.
3. Automatic list creation and whole-campaign activation are deliberately held. The owner creates and reviews those resources in the native UI.
4. Generic reconcile handles prepared outreach actions, not ambiguous contact provisioning. For an unknown preparation result, read the existing owned list and contact in Gojiberry. Preserve the original provider request ID when available. Do not repeat create_contact or fabricate a new contact ID.

Official sources: [Gojiberry API/MCP setup](#).

Local contract:

engine/channels/social.py:provision, send, send_existing_thread, reconcile.

5.17. Metadata

When: Optional creative and advertising extension. Parts 3.

Code support: Creative, audience/draft/read/pause supported; launch/restart currently held.

Entering it enables: Read the owned Metadata account, integrations and campaign status. Generate brand kit and creative. The portable extension can prepare audiences, offers, budgets and campaign drafts, plus exact native pause with separate advertising authority.

Limits: A token does not connect your ad accounts, fund spending or prove a running campaign. Launch and restart are held until existing campaign budget and native readiness contracts are verified. The original v5 source had creative only.

Credential: Account-scoped Metadata API token text.

Config, input or setup fields: credential_files.metadata,
settings.metadata_account_id, policy.advertising.enabled,
policy.advertising.max_daily_budget, policy.advertising.max_total_budget,
policy.advertising.authorized_campaign_ids.

Minimum account or credits: Your own Metadata account and token with read and creative permissions. The connection must return the exact configured account. No additional ad-platform key is needed for the local creative-only exercise.

Setup:

1. Sign in to your Metadata account. Open Settings, API Keys and create a named scoped token.
2. Save it privately. Set `credential_files.metadata` and your exact `metadata_account_id`.
3. Read `get_account_details`. Check account ownership before generation.
4. Use your own company domain. Generate/read the brand kit with `force_regenerate:false`.
5. Generate one creative with a short headline and specific instructions.
6. Inspect the actual image for logo, text, claims and layout. Save its generation receipt and review result.
7. For ads, connect your own actual channel accounts in Metadata. Read `get_integrations_status` and verify the required account permissions and resources.
8. Keep the advertising policy disabled until you approve the actual account, audience, copy and budgets. It is separate from the email authority.
9. Use `ads-review` with the exact operation and arguments. Inspect the resulting account and `approval_sha256`.
10. After explicit owner approval, use ads with the same permitted draft or pause operation, input and approved hash. Launch/restart remain held in the shipped adapter.
11. Read the named campaign after each write. Keep a queued or pending response separate from actual running status and spend.

Connection proof: Account read matches config. Creative has a completed generation receipt and a separate visual check. Advertising keeps the exact approved operation, native response and later campaign-status evidence. `provider_response_pending_review` is not a running campaign.

Provider wait: A creative response still needs visual review. For an unknown creative or advertising outcome, inspect the original attempt and owned Metadata UI. Campaign and ad name searches are read-only; a missing search result does not prove no object was created. Do not render, upload or create again.

Fallback: Create the same creative brief and review criteria locally. Mark generation unavailable when your account cannot call the tool.

Blank or structural example:

```
{
  "creative_input": {
    "operation": "generate_brand_creative",
    "arguments": {
      "domain": "",
      "headline": "",
      "instructions": "",
      "platform": "linkedin"
    }
  },
  "private_advertising_policy": {
    "enabled": false,
    "max_daily_budget": 0,
    "max_total_budget": 0,
    "authorized_campaign_ids": []
  },
  "campaign_operation_input": {
    "campaign_id": null
  }
}
```

Optional exported variable names: V5_METADATA_FILE, V5_METADATA_TOKEN.

Checks:

1. This API uses the bare Authorization token, without a Bearer prefix.
2. Optional campaign extension must display account, audience, ads, offer, exact budget and native readiness before any launch.
3. The portable campaign adapter is an addition. It does not establish that the original local v5 source already automated ad activation.
4. Each budget group must fit the authorized cap and request native monthly-cap pause. Each channel budget is checked. These argument checks are not an aggregate live-spend meter.
5. Read the exact tool schemas in METADATA-PUBLIC-TOOL-SCHEMAS.json before preparing inputs. The CLI input is the tool argument object, not a packet containing operation again.
6. Existing native campaigns or queued launches can continue independently. Local email pause does not stop them. Review and use the exact native campaign pause when necessary.
7. Campaign draft creation is limited to LinkedIn and Facebook under the captured budget contract. Google, Instagram, Microsoft, Reddit and unrecognized platform fields are held before a provider write. Creative-only generation is a separate operation.
8. Campaign draft creation accepts LinkedIn and Facebook channel fields only. Other channel fields remain held until their budget contracts are verified.
9. Generic reconcile does not cover creative, ad or audience/budget/offer provisioning attempts. Preserve the original request ID when returned, attempt key, exact operation, account and arguments. Read the existing object or owned Metadata UI before another write. Missing IDs remain missing.

Official sources: [Metadata token setup and scopes](#), [Metadata MCP transport](#).

Local contract: engine/assets.py:account, generate, advertising,
gtm.py:resource, ads-review, ads.

5.18. Local landing page and existing form

When: Optional publishing extension. Parts 1, 3.

Code support: Local page build and existing hosted/form reads supported.

Entering it enables: Create a local HTML page with the agent. Verify bytes from a recorded owned hosted URL and inspect an existing HubSpot form contract.

Limits: Local HTML is not a public deployment. A hosted content hash is not browser or form acceptance. Netlify publishing has its own supported entry below; Vercel and AWS publishing are not wired here.

Credential: No provider key consumed by the original v5 publish method.

Config, input or setup fields: settings.asset_hosts, settings.form_contract.

Minimum account or credits: No hosting account or paid API for a local HTML page. A published page needs your own hosting account. An existing embedded form needs its real portal, form ID and current source URL.

Setup:

1. Build and open the local page for your actual company and target account.
2. Check company facts, offer, proof and mobile layout.
3. Use your own existing form or booking link. Set settings.form_contract only for the supported HubSpot form contract.
4. When inspecting an existing hosted asset, use an owned host and a saved exact content hash.
5. For publishing a page through the portable runtime, follow the separate Netlify setup below.
6. Open the actual page on desktop and mobile. Submit an owner-controlled test form and inspect the native receipt.

Connection proof: Local page opens and displays your actual company context. Published bytes match the expected hash. Browser acceptance and form receipt are checked separately.

Provider wait: Hosting account approval, DNS and form setup may extend beyond class. A public URL alone is not proof that the form works.

Fallback: Keep the local page and exact publish brief. The attendee still leaves Part 1 with a usable page artifact.

Blank or structural example:

```
{
  "asset_hosts": [],
  "form_contract": {
    "source_url": "",
    "portal_id": "",
    "form_id": ""
  }
}
```


Checks:

1. The original source v5 publish method returns unavailable. The portable workshop package adds a bounded Netlify deployment method.
2. The form contract read checks fields and identity. It does not submit a form or establish acceptance.

Local contract: `engine/assets.py:inspect, form_contract, publish_file`.

5.19. Netlify: publish one HTML page

When: Required only for runtime page publishing. Parts 3.

Code support: Supported portable extension; original source had no upload.

Entering it enables: Publish one reviewed HTML file to your owned workshop site and verify the returned deployment and exact HTTPS bytes.

Limits: This uploads index.html for the whole selected site. Use a fresh workshop site. It does not add image files, submit your form, manage DNS or verify desktop/mobile usability.

Credential: Personal access token text.

Config, input or setup fields: `credential_files.netlify`, `settings.netlify_site_id`, `settings.netlify_owner_id`.

Minimum account or credits: An owned Netlify project and access token. The current Free plan permits API deployments with a monthly credit limit. Check your account before publishing; deployed traffic also consumes usage.

Setup:

1. Create a fresh Netlify workshop project under your own account. Do not select your existing company application.
2. In Applications, Personal access tokens, create a named token and save it privately.
3. Copy Project ID from Project configuration, General, Project details. Set `settings.netlify_site_id` and `credential_files.netlify`.
4. Let the agent read `/user` and `/sites/PROJECT_ID`. The code requires the site `user_id` to match the authenticated owner. Team sites that fail this check stay blocked.
5. Review the exact standalone HTML file. Keep images embedded or use approved public assets. Generate its SHA-256 with the full Python command in `SETUP-SEQUENCE.md`, then approve those exact bytes.
6. Run `resource --operation publish-review --file PRIVATE_PAGE.html`. Inspect its actual site, owner and content hash. After approving those exact values, use the returned `approval_sha256` with `publish`.
7. Read the same deployment ID until ready, then open its returned HTTPS URL. Run browser and owner-controlled form checks separately.

Connection proof: The owned site matches the authenticated account. The same native deployment is ready and its HTTPS page bytes match the approved SHA-256. Browser and form checks remain false until you perform them.

Provider wait: Post-processing is asynchronous. `accepted_pending_host` means accepted, not verified live. Rerun the same exact file/hash to read the existing deployment. An uncertain submission without a deployment ID must be resolved in Netlify before another upload.

Fallback: Open the local HTML file and keep the publishing brief. Continue the workshop without claiming a public page.

Blank or structural example:

```
{
  "credential_files": {
    "netlify": ""
  },
  "settings": {
    "netlify_site_id": "",
    "netlify_owner_id": ""
  }
}
```

Optional exported variable names: `V5_NETLIFY_FILE`, `V5_NETLIFY_TOKEN`.

Checks:

1. There is no automatic replay after an unknown upload result.
2. A deployment does not prove that the conversion form reaches your own CRM.
3. The approval hash binds site, owner and bytes. A content-only SHA-256 is useful to inspect the file but is not the publish approval hash.

Official sources: [Netlify token](#), [project ID and deployment API](#), [Netlify plan and credit limits](#).

Local contract: `engine/assets.py:publish_file`, `gtm.py:resource`.

5.20. Tavily: optional research alternative, not wired

When: Not required by this package. Parts 1, 2, 3.

Code support: No local adapter or CLI operation.

Entering it enables: Nothing inside this runtime. A separate explicitly connected coding-agent tool could use Tavily to find public sources.

Limits: Entering a Tavily key in this runtime will not activate research. There is no `V5_TAVILY` environment or config contract. Do not claim search, enrichment or email validation from a saved key.

Credential: No Tavily key consumed by the shipped runtime.

Config, input or setup fields: .

Minimum account or credits: No Tavily account or purchase needed for this workshop. Use existing agent web access or the supported Exa path.

Setup:

1. Use the supported official-source web path first.
2. If you deliberately add a separate Tavily tool, follow its official quickstart and keep its own key private.

3. Inspect the actual returned source page and save the supported claim. Do not treat this external tool as a shipped v5 adapter.

Connection proof: Not available from the shipped code. A separate tool needs its own actual search receipt.

Provider wait: No shipped provider call runs. Any separate integration has its own account and rate limits.

Fallback: Read the official company site with the coding agent or use the bounded Exa operation.

Blank or structural example:

```
{
"local_adapter": false,
"local_key_required": false
}
```

Checks:

- 1. Do not buy a second web-search tool to complete the same classroom exercise.

Official sources: [Tavily official quickstart](#).

Local contract: No Tavily operation in engine/research.py or tools/setup.py.

6. What a key cannot solve

Outcome	Current package status	What remains
Desktop/mobile page and form acceptance	manual live check required	Portable Netlify verifies uploaded HTML bytes. Open the live page and inspect an owner-controlled form receipt separately.
Postal direct mail	not wired	Loop & Tie is email gift delivery. No postal fulfillment provider is implemented.
Calendly scheduling writes	not wired	Bookings can be read. This adapter does not book a meeting.
Exact attendance association	partial	HubSpot meeting rows are read; attendee attendance stays unknown without an exact contact/email/start join.
Unattended company source coverage	native setup and live proof required	Strict native mode now exists. The company must supply complete maintained customer, CRM, full history and booking sources and verify their actual fresh receipts.
Ad launch/restart	deliberate runtime hold	Audience/draft/creative/read/pause operations remain. Automatic launch/restart waits for verified existing-campaign budget and readiness contracts.

An adapter that exists is not a verified live result. The attendee must connect their own account and inspect the native outcome.

7. Completion checks

1. Your page opens with your actual company facts and offer.
2. Your mailbox identity and primary-calendar read match your private config.
3. Your exclusions come from a complete current source, with no invented empty coverage.
4. Every selected person has current employer evidence and fresh valid work-email evidence.
5. The eleven exact pilot actions are pinned. The own-sender Sent test unlocks only the ten approved prospect actions.
6. Every claimed send has an exact native Sent match. Prepared, accepted and queued stay separate.
7. The second cycle reads prior attempts and does not submit the same uncertain or completed action again.
8. Gift and social extensions have their own provider readbacks. Recipient waits stay visible.
9. Unimplemented outcomes are named. They are not counted as completed because a key was entered.

8. Unknown provider outcomes

For exact email, gift and social outreach keys, use reconcile. Creative, advertising and contact preparation do not have that generic reconciliation route. Follow PROVIDER-RECOVERY.md. Preserve original request IDs when returned and never submit a duplicate to learn whether the first request worked.

Loop & Tie pricing source: [What are Loop & Tie Credits?](#). The provider collection example shows price without a currency field: [Generating Gift Links](#).